Gait-Phase-Aware Command Timing for Decentralized Quadruped Swarms

Bishwaswarup Nayak — Indian Institute of Science, Bengaluru, India · bishwaswarup@iisc.ac.in (corresponding author)

Abstract

Multi-robot collision avoidance treats each robot as a point that can change velocity at any instant. A trotting quadruped cannot: how well it absorbs a change of commanded velocity depends on where in the gait cycle the change arrives. We measure this dependence for a Bezier-curve trot controller on a simulated Unitree Go1 and use it in a decentralized swarm. The phase-aware layer solves reciprocal collision avoidance (ORCA) one gait period ahead, releases each new velocity command at the gait phase our capability map predicts is least destabilizing, and shapes it with a gait-aware slew limiter. Urgent conflicts bypass the timing.

In MuJoCo, across 4 to 32 robots in antipodal swaps, pillar fields, a narrow gate and a 53 m obstacle course, the full system is compared with the same stack without timing (paired by seed, Holm-corrected). In crowded and obstructed scenarios it cuts urgent avoidance events by 22-39% and 99th-percentile body tilt by 19-24%. It increases the closest robot-robot distance in formations by 2-5 cm (4-8%), and the full system had no falls in any run. Prediction without phase timing gives no consistent benefit, so the timing is the active ingredient. In sparse traffic (4 robots) the method costs more than it saves. All code and data are open source.

Keywords: legged robots, multi-robot systems, collision avoidance, gait phase, quadruped swarms, ORCA

I. Introduction

Decentralized collision avoidance methods such as reciprocal velocity obstacles and ORCA [1], [2] give each robot a new velocity every control tick. They assume the platform can follow that velocity, possibly with an acceleration limit. Wheeled robots and drones come close to that assumption. A trotting quadruped does not: its body is supported by two diagonal feet at a time, the support line rotates during each stance, and a velocity change that arrives at the wrong moment produces a large body tilt or a stumble.

That a legged gait responds to a disturbance differently at different phases is well known in single-robot locomotion: push recovery, capture-point and step-timing work all exploit it [5]-[8]. What has not been studied, to our knowledge, is the consequence for multi-robot navigation. A swarm layer that issues frequent avoidance commands is a source of disturbances to every robot's gait. Formation and swarm controllers for quadrupeds [3], [4] shape *what* each robot is commanded, but not *when* within the gait the command is delivered.

This paper asks one question: does timing the swarm layer's velocity changes to the gait phase make a quadruped swarm safer, and at what cost? We answer it in simulation with a fully open stack and paired statistics. Our contributions are:

1. **A measurement of phase-dependent fragility** for a Bezier trot on a Unitree Go1 model: a pilot experiment, a capability map over phase, direction and size of a command change (held-out R^2 0.94-0.98 vs 0.70-0.87 phase-blind), and a push-impulse envelope in which knowing the phase raises the tolerable impulse 1.33-1.44×.
2. **A phase-aware command layer**: a commander that releases each velocity change at the predicted best phase within a deadline, and a gait-aware slew limiter that shapes the change by phase.
3. **A predictive ORCA layer** that plans one gait period ahead, so the commander has time to wait for a good phase, with an urgent bypass when time-to-collision is short.
4. **An ablation and scaling study** over 4-32 robots, a gate, pillar fields and a 53 m obstacle course (790 runs in the final configuration, paired by seed, Holm-corrected), including the negative results: where the method does not help and which robustness fixes we tried and rejected.

All code, data and scripts are released under the MIT licence.

II. Related Work

Reciprocal collision avoidance. Velocity obstacles and their reciprocal forms [1], [2] are the standard decentralized method for crowds of robots; ORCA reduces each tick to a small linear program. Extensions add platform limits: acceleration-velocity obstacles bound the acceleration [9], and NH-ORCA bounds the tracking error of non-holonomic robots [10]. All of these model the platform's limits as constant in time. Our work keeps ORCA unchanged and instead models a limit that varies periodically with the gait.

Multi-quadruped formation and navigation. Recent quadruped-team work uses distributed MPC with control barrier functions [3] or leader-follower formations with path planning [4]. These treat each quadruped as a velocity-controlled point and rely on the locomotion controller to follow the commands. We do not compete with these planners; the phase-aware layer can sit between any of them and the locomotion controller.

Phase-dependent response of legged gaits. The capture point [5], step-timing adaptation [6] and switched-system capturability for quadrupeds [7] all show that what a legged robot can recover from depends on its state within the step. Gait-sensitivity analysis [8] measures how disturbances propagate through a limit cycle. These results are used to choose footholds and step times for one robot. We use the same fact one level up: to choose *when* the navigation layer changes its command.

Command shaping for legged robots. Velocity commands to legged controllers are commonly rate-limited, and learned controllers are trained on curricula of command changes. A recent example schedules commands by the robot's current velocity to reach high speed [11]. To our knowledge, no prior work schedules navigation commands by gait phase.

III. The Low-Level Controller and Its Phase-Dependent Fragility

Controller. Each robot runs the same open-source trot controller. Feet follow a 12-point Bezier swing curve [12], [13] in a diagonal trot (period 0.36 s, duty factor 0.5, swing clearance 0.07 m, hip-to-foot height 0.27 m). A PI loop on body velocity ($k_i = 1.5$) removes steady drift, Raibert foot placement [14] (gain 0.1) handles velocity errors, and joint targets go to the position actuators of the MuJoCo Menagerie Go1 model [15], [16]. A slew limiter bounds the rate of change of the command at (1.5 m/s², 1.0 m/s², 3.0 rad/s²) for (v_x, v_y, ω_z). The controller tracks commands within about 0.02 m/s, survives 4 minutes of random command changes without a fall, and walks at heights from 0.16 m to 0.34 m.

Stability. Treating one gait period as a stroboscopic Poincaré map, the largest Floquet multiplier is 0.42-0.59 for the full controller and 0.73-0.87 open loop, so the gait is orbitally stable across the commanded speed range. Stability of the cycle does not say how a single command change is absorbed, which depends on when it arrives.

Why the phase matters. In a trot the body is supported by one diagonal pair at a time: front-right with rear-left, then front-left with rear-right, at roughly −34° and +34° to the heading for the Go1's geometry. Within one stance the effective support axis rotates from about +25° to +65° as the body moves over the feet. A change of commanded velocity that pushes the body across the current support axis is resisted by the stance legs; one that pushes along it tips the body about that axis. The same change therefore produces different tilt at different phases, and by the mirror symmetry of the trot the effect repeats every half period with the sign of the lateral direction flipped.

Measured fragility. A pilot experiment applied command steps and body pushes at controlled phases. Phase had a large effect: in 27 of 64 push conditions, whether the robot recovered depended on the phase alone. From these data we built a capability map: a lookup table, periodic in phase and symmetrized, that predicts peak tilt and fall risk for a command change of a given direction and size at a given phase. On held-out trials it explains 94-98% of the variance (R^2), against 70-87% for the same model without phase. A push-impulse envelope found by bisection agrees with the pilot in 98.6% of cases, and knowing the phase raises the impulse the robot tolerates by a factor of 1.33-1.44.

What this means for a swarm. In isolated step tests, releasing a command at the best phase strictly dominated the slew limiter alone. In a stream of swarm commands, however, timing alone did not work: only 0-2% of the commands resembled the settled single steps the map was measured on, and about half were urgent, leaving no time to wait for a good phase. Section IV is designed around that finding: the swarm layer must look ahead so that most changes are known before they are needed.

IV. Method

Every robot runs the same four-stage stack at 10 Hz, with no central planner and no communication beyond the neighbour positions and velocities ORCA needs. A safety layer that is identical in every configuration, the baseline included, wraps the stack.

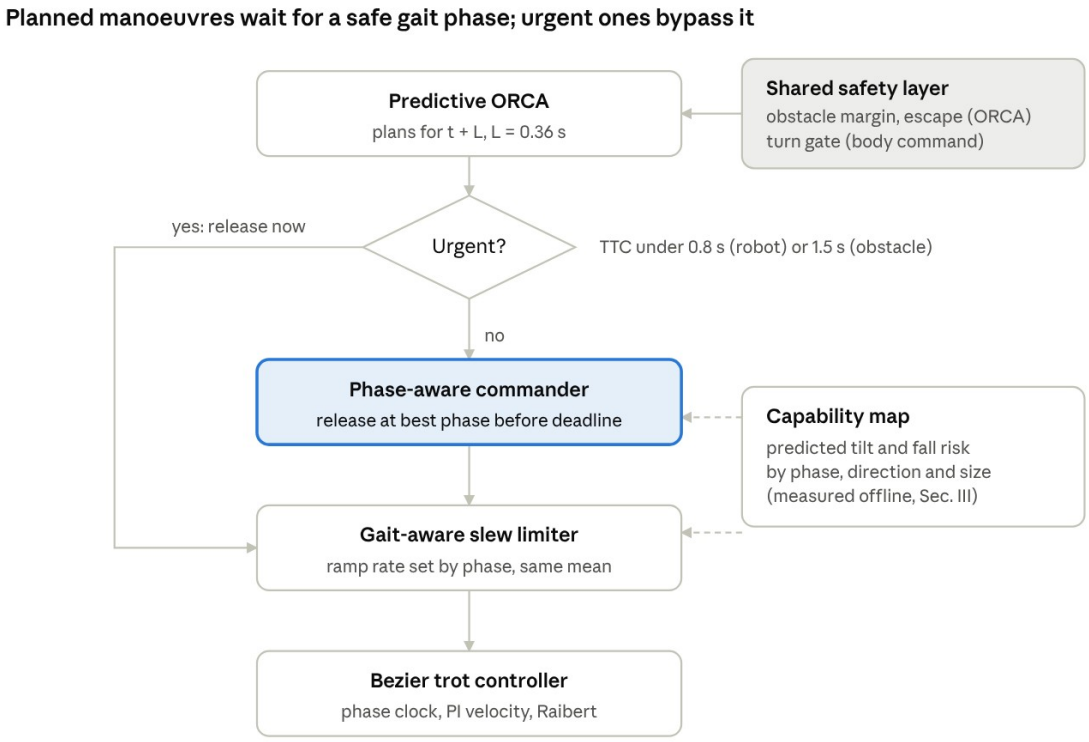


Fig. 1 · per-robot stack at 10 Hz

Fig. 1. Each robot's stack. ORCA plans one gait period ahead; non-urgent changes wait in the commander for the phase the capability map predicts is safest, urgent ones go straight to the limiter. The safety layer is the same in every configuration.

A. Predictive swarm layer. Each robot is a disc of radius 0.38 m with maximum speed 0.6 m/s. Rather than solving ORCA for the present, the robot solves it for the situation one lookahead L ahead (L = 0.36 s, one gait period). Its own position is extrapolated with the command it is currently executing, and its neighbours with their measured velocities. The result is the velocity the robot will need at time t + L, so it can be requested with a deadline of L. If the current command would collide within 0.8 s with a robot, or within 1.5 s with a static obstacle, the request is urgent: ORCA is solved for the present and the command is released at once. A 15° keep-right bias breaks symmetric deadlocks.

B. Phase-aware commander. A request whose change from the current command exceeds 0.15 m/s (or 0.3 rad/s) is a manoeuvre. For each manoeuvre the commander queries the capability map for every gait phase reachable before the deadline (at most 0.5 s). It releases the command at the phase with the lowest predicted cost: peak tilt plus a weight of 50 on predicted fall probability. Smaller changes, urgent requests and changes outside the map's domain are released at once. A new request similar to a pending one keeps the pending release time, so the timing is not reset every tick.

C. Gait-aware slew limiter. Once released, a change is applied through a slew limiter whose rate depends on phase. The limiter is built from the capability map: it ramps faster in phases where the map predicts little tilt for the current direction of change and slower where it predicts a lot. Its rate is scaled

so that its mean over one period equals the uniform limiter's, so it does not gain speed by being more aggressive on average.

D. Formation and queueing. Formations follow a reference point with slot offsets that shrink laterally when obstacles are close ahead. In bottlenecks, the robot further back takes 90% of the avoidance responsibility in ORCA and the one in front 10%, so robots queue instead of shouldering through.

E. Shared safety layer. In every configuration, ORCA sees static obstacles inflated by 0.1 m with a 2 s horizon. Body commands are clipped to the range in which the gait was stress-tested ($|\omega_z| \le 1.0$ rad/s, $|v_y| \le 0.3$ m/s), and forward speed is scaled from 100% to 30% as the heading error grows from 30° to 90°, so a robot turns before it walks. If ORCA returns less than 0.05 m/s for more than 1 s while the robot wants to move, the preferred velocity is turned 75° towards the formation reference for 2 s, and to the other side if the robot is still stuck. Each of these was added after a failure in the experiments of Section VI; Section VII reports one further fix that we tried and rejected.

V. Experimental Setup

All experiments run in MuJoCo with N copies of the Menagerie Go1 model in one scene (foot friction 0.9, shared meshes). The swarm layer runs at 10 Hz and the gait controller at the physics rate. Each seed perturbs every start position by 5 cm (standard deviation) and gives every robot a random initial gait phase, so robots are never phase-locked.

Scenarios.

Scenario	Robots	Layout	What it tests
swap4 / 8 / 16 / 32	4 / 8 / 16 / 32	On a circle of radius 2.5 / 4 / 6 / 9 m, each robot walks to the opposite point	Robot-robot crossing traffic; density grows with N
formation9_pillars	9 (3×3, 1.4 m spacing)	Formation travels 14 m at 0.35 m/s through 5 pillars (r 0.3 m)	Formation through clutter
formation6_gate	6 (2×3)	Wall with a 1.9 m opening	Bottleneck: team must narrow and queue
formation16_pillars	16 (4×4)	18 m through 9 pillars	Dense formation through clutter
Obstacle course	1 and 6	53 m path at 0.45 m/s: pole slalom, 1.9 m gate, chicane, two moving obstacles timed to cut across the team (0.4 m/s), 7 pillars	Everything in sequence

Configurations. The ablation adds one component at a time. All share the safety layer of Section IV-E.

Configuration	Prediction	Phase timing	Gait-aware limiter	Queueing
Baseline (reactive)	–	–	–	–
Queueing only	–	–	–	yes
Prediction only	yes	–	–	–
Prediction + timing	yes	yes	–	–
Prediction + timing + limiter	yes	yes	yes	–
Full system	yes	yes	yes	yes

The main suite runs all six configurations on swap4, swap8, formation9_pillars and formation6_gate with 20 seeds (480 runs). The scaling suite runs the baseline, prediction only (with queueing) and the full system on swap16, swap32 and formation16_pillars with 10 seeds (90 runs). The lookahead suite varies L over 0.18, 0.36, 0.54 and 0.72 s on swap8 and the gate (10 seeds, 100 runs). The course runs three configurations for teams of 1 and 6 with 20 seeds (120 runs).

Metrics. Three metrics are fixed in advance as primary: urgent avoidance events per robot-minute (how often a robot had to change course with less than 0.8 s to spare), the 99th-percentile body tilt over the run, and the closest robot-robot distance. We also report falls (trunk below 0.15 m or tilted more than 60°), runs not finished within the time limit, contacts, and completion time.

Statistics. Configurations are compared with the baseline on the same seed. For each metric we take the per-seed difference and test it with a two-sided sign-flip permutation test (exact for up to 16 seeds, 20,000 random sign patterns otherwise), with 95% bootstrap confidence intervals. The 21 primary tests (7 scenarios × 3 metrics, full system vs baseline) are corrected together with the Holm method. With 10 seeds the smallest attainable corrected p-value is about 0.02, which limits what the scaling suite can show.

VI. Results

The full system reduces urgent events and tilt wherever robots crowd or obstacles force manoeuvres, and does no good in sparse traffic. Of the 21 primary tests, 15 are significant after Holm correction: 13 favour the full system and 2, both in swap4, favour the baseline.

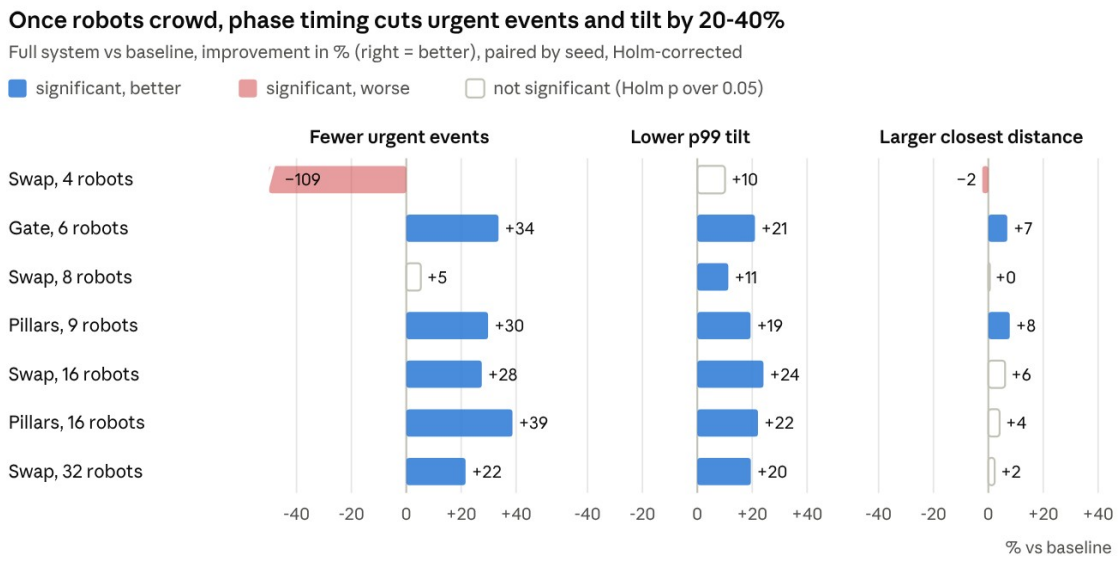


Fig. 2 · results/paper/primary_tests.csv · final --robust runs, 10-20 seeds per scenario

Fig. 2. Improvement of the full system over the baseline on the three primary metrics. The swap4 bar for urgent events is cut at −50% (true value −109%).

A. Which component does the work. Prediction alone does not help. Against the baseline it changes urgent events by +6% in swap8 and +3% in the pillar field, and raises 99th-percentile tilt by 20% in swap4. Adding phase timing produces most of the gain: urgent events fall by 15-16% and tilt by 12-16% in swap8, the pillars and the gate. The gait-aware limiter adds further tilt reduction at the gate (−15% instead of −12%) but almost none elsewhere. Queueing matters in obstructed formations, where it doubles the reduction in urgent events (pillars −30% instead of −18%, gate −34% instead of −14%) and widens the closest distance by 7-8%. Phase timing is therefore the active ingredient, and queueing is a useful complement only where robots must pass through gaps.

B. Scaling. The benefit holds as the team grows. With 16 robots in a swap the full system cuts urgent events by 28% and tilt by 24%; with 32, by 22% and 19%; with 16 robots in a formation through pillars, by 39% and 22%. The closest-distance gain shrinks at 32 robots (+2%, not significant), where the swarm is near its packing limit.

C. Cost. Waiting for a good phase costs time when there is nothing to avoid. In swap4 the full system is 6.8% slower, has twice the urgent events of the baseline (5.2 to 10.9 per robot-minute) and a 2% smaller closest distance. In swap8 it is 2.7% slower; at the gate the completion time is unchanged.

D. Lookahead. Looking further ahead lets more changes be planned: the share of manoeuvres that are urgent falls from 56% (baseline) to 43%, 41%, 25% and 22% for L = 0.18, 0.36, 0.54 and 0.72 s in swap8. But the number of command changes per robot-minute doubles from 54 to 112, because predictions made further ahead are revised more often. Tilt is about equally reduced at every L (swap8: 6.1-6.6° against 7.5°; gate: 4.4-4.8° against 5.9°). One gait period (0.36 s) is the setting we use; 0.54 s is slightly better at the gate but doubles command churn in swaps.

E. Obstacle course. For the team of six, the full system lowered 99th-percentile tilt from 7.95° to 6.68° (−16%), the worst tilt over 20 runs from 18.3° to 13.7°, urgent events by 14% and robot-robot contacts from 0.25 to 0.05 per run, at the same completion time (129 s). One of its 20 runs did not finish in time; the baseline had one fall. For a single robot the three configurations are nearly identical (tilt 4.7-4.9°), and prediction raised urgent events (7.4 to 10.4 per minute), consistent with the swap4 result.

F. Safety totals. Across all 230 full-system runs in the final configuration there were no falls; the baseline fell twice in 170 runs (once in the 16-robot pillar field, once on the course). Runs that did not finish in time were rare in every configuration: 1 for the full system (course), 1 for queueing only, 2 for prediction only.

VII. Discussion and Limitations

When it helps. Phase timing pays off when a robot has many manoeuvres to make and enough warning to wait for a good phase. That is what crowds and clutter provide and what sparse traffic does not. In swap4 the few manoeuvres are mostly avoidable by going slightly earlier, and any waiting only delays them, which shows up as more urgent events later. A practical system should enable timing only above a traffic density threshold; we have not tested such a switch.

Robustness fixes and a rejected one. The shared safety layer of Section IV-E was not designed in advance. Each part was added after a failure found by the experiments: robots falling after combined sideways walking and fast turning (turn gate), robots pinned against a wall between them and their slot (escape), and robots grazing pillars (static margin). Because the layer is identical for every configuration, it does not favour the full system. We also tried a stronger escape rule, which detects a stall by lack of progress instead of low speed and gives late robots right of way. It removed every fall, but runs that did not finish rose from 1 to 16 of 480 in the main suite, because it triggered during normal avoidance and sent robots on long detours. We report it as a negative result; the code keeps it as an option.

Limitations. (1) All results are in simulation, with one robot model and one trot controller; the capability map must be re-measured for another controller, and the effect size on hardware is unknown. (2) The capability map is built from single command changes in settled walking; most swarm commands fall outside that domain, which is why prediction and a shaped limiter are needed alongside timing. (3) Robots share position and velocity perfectly, without noise or delay. (4) The scaling suite has only 10 seeds, which limits the attainable significance. (5) Completion time rises by up to 7% in sparse scenarios.

VIII. Conclusion

A quadruped's tolerance of a velocity change depends on where in the gait cycle it arrives, and a swarm layer can use this. Planning ORCA one gait period ahead and releasing each change at the predicted best phase cut urgent avoidance events by 22-39% and 99th-percentile tilt by 19-24% in crowded and obstructed scenarios with 6 to 32 simulated Go1 robots, with no falls in 230 runs. Prediction without timing gave no consistent benefit. Next steps are hardware experiments on Go1 robots, a second low-level controller (a learned policy) to test whether the capability map transfers, sensing noise and delay, and an automatic switch that disables timing in sparse traffic.

Declarations

Funding. This research received no external funding.

Competing interests. The author declares no competing interests.

Data and code availability. All code, configuration files, raw results and analysis scripts are available at github.com/Bishwaswarup/robot under the MIT licence. The final results tables are produced by `scripts/paper_tables.py`.

Author contributions. Bishwaswarup Nayak conceived the study, developed the software, ran the experiments, analysed the results and wrote the manuscript.

Use of AI tools. Claude (Anthropic) was used to assist with software development, experiment scripting, data analysis and drafting of the manuscript. The author reviewed and verified all code, results and text and takes full responsibility for the content.

Ethics approval. Not applicable.

References

[1] J. van den Berg, S. J. Guy, M. Lin, D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research* (ISRR 2009), Springer STAR vol. 70, 2011, pp. 3-19.

[2] J. van den Berg, M. Lin, D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *Proc. IEEE ICRA*, 2008.

[3] W. Zhan, Z. Liang, H. Song, W. Pan, "Safe distributed learning-enhanced predictive control for multiple quadrupedal robots," arXiv:2503.05836, 2025.

[4] M. Norén, M.-N. Stamatopoulos, A. Banerjee, G. Nikolakopoulos, "Role-adaptive collaborative formation planning for team of quadruped robots in cluttered environments," arXiv:2602.18260, 2026.

[5] J. Pratt, J. Carff, S. Drakunov, A. Goswami, "Capture point: A step toward humanoid push recovery," in *Proc. IEEE-RAS Humanoids*, 2006.

[6] M. Khadiv, A. Herzog, S. A. A. Moosavian, L. Righetti, "Step timing adjustment: A step toward generating robust gaits," in *Proc. IEEE-RAS Humanoids*, 2016, arXiv:1610.02377.

[7] H. Chen, Z. Hong, S. Yang, P. M. Wensing, W. Zhang, "Quadruped capturability and push recovery via a switched-systems characterization of dynamic balance," arXiv:2201.11928, 2022.

[8] D. G. E. Hobbelen, M. Wisse, "A disturbance rejection measure for limit cycle walkers: The gait sensitivity norm," *IEEE Trans. Robotics*, vol. 23, no. 6, 2007.

[9] J. van den Berg, J. Snape, S. J. Guy, D. Manocha, "Reciprocal collision avoidance with acceleration-velocity obstacles," in *Proc. IEEE ICRA*, 2011.

[10] J. Alonso-Mora, A. Breitenmoser, M. Rufli, P. Beardsley, R. Siegwart, "Optimal reciprocal collision avoidance for multiple non-holonomic robots," in *Distributed Autonomous Robotic Systems* (DARS 2010), Springer, 2013.

[11] Y. Tao, S. Cheng, G. Lan, Y. Yuan, Y. Jin, H. Wang, "Extending the speed limit of quadrupedal locomotion via refined actuator modeling and adaptive command scheduling," arXiv:2609.13289, 2026.

[12] D. J. Hyun, S. Seok, J. Lee, S. Kim, "High speed trot-running: Implementation of a hierarchical controller using proprioceptive impedance control on the MIT Cheetah," *Int. J. Robotics Research*, vol. 33, no. 11, 2014.

[13] M. Rahme, "spot_mini_mini," GitHub repository (MIT licence), 2020.

[14] M. H. Raibert, *Legged Robots That Balance*. MIT Press, 1986.

[15] E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ IROS*, 2012.

[16] K. Zakka et al., "MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo," GitHub repository, 2022.

Code and data: github.com/Bishwaswarup/robot (MIT licence).